

AI Business Analytics System
Internship Project Report
Team Members : Shajahan S , Jeberson J

1. Abstract

The AI Business Analytics System is a full-stack, machine-learning-powered web application designed to transform raw business sales data into actionable, real-time insights. By leveraging Python-based predictive algorithms and an intuitive glassmorphic dashboard, the system provides business decision-makers with automated sales forecasting, intelligent customer segmentation, and real-time metric tracking. To improve accessibility, the system integrates a Natural Language Processing (NLP) chatbot that allows users to query their data conversationally. The final software is deployed onto a fast, highly-available Serverless architecture via Vercel.

2. Introduction

In standard enterprise environments, parsing through millions of rows of CSV or database records can be sluggish and requires extensive data analysis expertise. Small to medium businesses often lack the resources to maintain data engineering teams.

Objectives:

- Automate data analysis using machine learning algorithms.
- Provide a zero-latency, real-time dashboard tracking Key Performance Indicators (KPIs).
- Implement an AI Chatbot to democratize data retrieval.
- Ensure the application is responsive, lightweight, and deployment-ready.

3. Technology Stack

The project was developed using a lightweight, highly decoupled Client-Server architecture:

- * **Frontend Interface:** HTML5, Vanilla JavaScript, CSS3 (Custom Glassmorphism UI)
- * **Data Visualization:** Chart.js
- * **Backend API Server:** Python 3, Flask framework
- * **Machine Learning Core:** scikit-learn (Linear Regression, K-Means), Pandas, NumPy
- * **Database Integration:** CSV (Local Storage) & Supabase (PostgreSQL Cloud)
- * **Deployment:** GitHub (Version Control), Vercel (Serverless Edge Functions)

4. System Architecture

The platform is separated into a frontend visualization layer and a backend computational layer.

4.1 Serverless Cloud Deployment

The traditional method of hosting Python servers requires always-on background threads. To modernize the project for the internet scale and reduce costs, the backend was adapted to run on ****Vercel Serverless Functions****. The system operates statelessly, processing data and executing Machine Learning inferences on the fly precisely when an endpoint (such as ``/api/analytics`` or ``/api/live_event``) is triggered by the frontend.

4.2 Real-time Simulation Engine

Since real company sales streams are typically private, the backend incorporates a pseudo-real-time simulator. Rather than using an infinite loop, the Vercel-hosted API handles spontaneous HTTP polling requests from the dashboard, dynamically generating realistic transactional data payloads (mocking regions, customer counts, and profit margins) and seamlessly feeding them into the frontend without forcing page reloads.

5. Machine Learning Methodology

The core value of the application rests in its predictive algorithms implemented via `scikit-learn` in `backend/ml\_model.py`.

5.1 Sales Forecasting (Linear Regression)

To predict short-term revenue, the application uses ****Simple Linear Regression****.

1. ****Data Preprocessing:**** Historical data is ingested via Pandas. The `date` column is converted into numerical ordinals representing the number of days since the earliest recorded local epoch.
2. ****Training:**** The model is trained dynamically using the `(date, sales)` coordinate mappings.
3. ****Inference:**** Using equations derived from the line of best fit, the system outputs the projected revenue for the next 7, 30, or 90 days.

5.2 Customer Segmentation (K-Means Clustering)

To identify high-value demographics, the system utilizes the unsupervised ****K-Means algorithm****.

1. The features `customers` and `profit` are extracted into a matrix.
2. The algorithm partitions the data into 3 distinct clusters (e.g., standard, developing, and premium regions).
3. The coordinates are normalized and assigned to cluster centers, providing immediate marketing insight on which region generates the highest profit per customer.

6. Natural Language Chatbot

A custom NLP ruleset was engineered in `backend/chatbot.py`. Rather than relying on expensive LLM generation tokens like OpenAI, this chatbot securely scans keyword intents (e.g., "sales", "profit", "best day") using Python string evaluations. Once parsed, the chatbot hooks directly into the ML aggregate functions to retrieve live database calculations and formats them into conversational language on the frontend.

7. User Interface (UI) and UX

- * ****Aesthetic:**** The design language utilizes "Glassmorphism" – an overlapping semi-transparent frosted styling built with pure CSS backdrop filters over a dark-mode mesh gradient.
- * ****Responsiveness:**** Native `@media` queries dynamically scale grids down vertically, allowing 100% functionality on ultra-narrow mobile screens. A mobile hamburger drop-down ensures the menu does not clash with the charting canvases on 480px viewports.
- * ****Micro-interactions:**** Incoming live events trigger smooth `ease-in` slide animations and brief CSS flashes on KPI cards, alerting the user to real-time changes instantly.

8. Conclusion

The AI Business Analytics System successfully proves that machine learning models can be seamlessly integrated into interactive, lightning-fast edge applications. By transitioning the infrastructure to a

serverless model and coupling it with vanilla JavaScript API polling, the project delivers enterprise-grade insights with zero technical overhead for the final user.

Generated for Academic Documentation & Project Submission.